SSD Performance Profiling on an Apple M5 Max

ECSE 4320/6320 Class-wide Project 3

October 2026

Summary

This report profiles the internal SSD of an M5 Max MacBook Pro (APPLE SSD AP2048Z, 2 TB, controller inside the SoC) through one 32 GiB file on APFS with direct I/O. **Measured:** 4 KiB random read at queue depth 1 takes 55 to 70 µs; reads reach 12.9 GB/s and writes about 10.6 GB/s; 4 KiB random reads saturate at 189 k to 223 k IOPS with the knee at QD16, while 4 KiB random writes reach only 47 k IOPS. A 2.2 TB sustained sequential write held 11.0 GB/s with no cliff, a result limited by the test and not proof that no SLC-style cache exists. A page-cache run inflates IOPS 90 times (1.32 M against 14.5 k). The same read workload ran at two speed levels (about 189 k and 222 k IOPS) for a reason that could not be established. Predictions written before data collection are scored in Section 9, including the misses. Claims are labeled **measured**, **inferred** or **hypothesis**.

1. Methodology

Item	Value
Platform	MacBook Pro, Apple M5 Max (18 cores: 6 Super + 12 Performance, no efficiency cores), 36 GiB RAM, macOS 27.0.1. The OS was upgraded from 26.5.2 mid-project; precondition and all experiments ran on 27.0.1.
SSD	APPLE SSD AP2048Z, 2 TB, NVMe-class, controller integrated in the SoC, TRIM supported, wear 0%. PCIe link generation and width cannot be read on macOS.
Filesystem	A file on APFS (4 KiB blocks), not a raw device. FileVault is on, so hardware encryption and copy-on-write are part of every number. Logical file offsets are not device LBAs.
fio and engines	fio 3.43. Available: `psync` (and other synchronous engines), `posixaio`, `mmap`. Not available on macOS: `libaio`, `io_uring` (detected at run time and recorded). Fallback: `psync` with `numjobs=N`, and `posixaio` with `iodepth=N`.
Queue depth	`psync` depth = `numjobs` (fio supports only about 16 jobs here: macOS limits System V shared memory to 4 MB). `posixaio` reaches depth 256 in one process on macOS 27.0.1 (`kern.aioprocmax` was 16 on 26.5.2). Effective depth is checked for every run with IOPS x mean latency.
Test file and range	One 32 GiB file in a Spotlight-excluded directory. Offsets span 0 to 32 GiB (1 GiB for the range test), always block-aligned.
Data pattern	`refill_buffers=1`, `buffer_compress_percentage=0`: incompressible, fresh random data per IO, identical for every job.
Preconditioning	File written once sequentially (4 jobs, 1 MiB blocks, 32.0 GiB at 6.2 GB/s) before any read test, so reads hit allocated blocks.
Power, thermal, load	AC power, Low Power Mode off, battery held at 80% by macOS. SMART temperature, wear, free space, `pmset` state and top CPU consumers logged before and after every stage. Desktop apps were running at a few percent CPU, so the machine was not perfectly idle.

Item	Value
CPU controls	SMT: none (Apple silicon; 18 physical and 18 logical cores, 6 Super and 12 Performance, no efficiency cores). Frequency governor: not exposed to the user on macOS; Low Power Mode off (`pmset` reported powermode 0 in every stage log); actual CPU clocks during runs were not observable. Pinning and isolation: fio reports that CPU affinity is not supported on this platform, so there was no thread pinning and the macOS scheduler placed the fio threads; the machine was idle apart from the background load noted above.
Repetitions and timing	3 trials per point (mean and sample sd), except Exp 6b (one run, capped by the team). Runtime / ramp: Exp 1 30 s / 5 s; Exp 2 reads 15 s / 2 s, writes 4 s / 1 s (shortened to limit wear); Exp 3, 4, 6a 20 s / 3 s; Exp 5 60 s / 5 s; Exp 7 20 s / 5 s.
Validity guards	Before every run: AC power required. After every run: result moved to a quarantine folder and the stage aborted if the Mac left AC power or the run took longer than runtime + ramp + 45 s. Each stage runs under `caffeinate`. Free space after the file must stay above 20%; every fio target must resolve inside the scratch directory.

Direct I/O verification. `F_NOCACHE` itself was not traced (that needs sudo and relaxed SIP), so the evidence is behavioral: (1) on the same file and 1 GiB range, QD1 4 KiB random read gave 61.4 µs with `direct=1` and 0.70 µs with `direct=0` (87 times lower), and the check script stops the whole campaign if `direct=1` looks cache-like; (2) the SSD's own SMART "Data Units Read" counter never fell below fio's byte count (ratio 1.11 to 1.55, the excess being ramp-time IO that fio does not count; the 1.55 for Exp 3 and 4 includes writes and is unexplained beyond ramp time); (3) fio's written bytes equal the SMART "Data Units Written" delta to 0.01% (2.199 TB each in Exp 6b).

Quarantined runs. The first pass of the Exp 2 write runs is invalid: the charger was unplugged and the lid closed, and the Mac slept repeatedly during the runs (`pmset` log), producing single writes of up to 180 s. The 42 files are kept untouched in a quarantine folder in the repository, excluded from all results, and the runs were repeated under the guards above.

2. Exp 1: zero-queue baselines (QD1)

Hypothesis. 4 KiB random read 60 to 90 µs; 4 KiB random write 20 to 60 µs and faster than read; 128 KiB sequential read 70 to 150 µs; 128 KiB sequential write 80 to 200 µs; p99 at 1.5 to 3 times p50.

Table 1. Exp 1 results. Fixed: QD1, 1 job, 32 GiB range, direct I/O. Varied: workload. Mean (sd) over 3 trials.

workload (QD1, 1 job)	IOPS	MB/s	mean µs	p50 µs	p95 µs	p99 µs
4 KiB random read	18,279	75	54.6 (0.5)	54.0 (0.0)	58.8 (0.6)	62.7 (1.8)
4 KiB random write	12,319	50	80.9 (3.5)	69.8 (3.1)	91.6 (3.1)	181.2 (46.1)
128 KiB sequential read	7,403	970	134.9 (0.1)	131.6 (0.9)	179.2 (0.0)	206.5 (1.2)
128 KiB sequential write	19,352	2,536	29.1 (0.7)	28.7 (0.6)	35.1 (1.0)	39.5 (0.6)

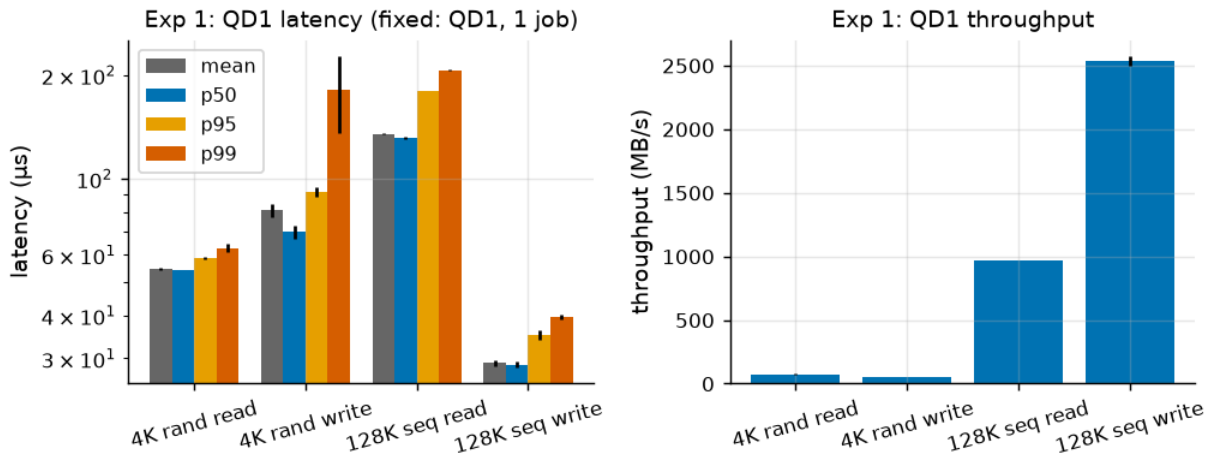


Figure 1. Exp 1: QD1 latency percentiles (log scale, µs) and throughput (MB/s). Fixed: QD1, 1 job. Varied: workload.

Result. Random read (54.6 µs) is below the predicted range and random write (80.9 µs) is **slower** than read, the opposite of the prediction. Sequential read matches (134.9 µs, 970 MB/s). Sequential write is far below the range (29.1 µs per 128 KiB). The random-read tail is much tighter than predicted (p99/p50 = 1.16); random-write p99 is 2.6 times p50, as predicted.

Interpretation. *Inferred:* a QD1 read costs one flash page read plus controller, encryption, PCIe and kernel time, and nothing overlaps; a 128 KiB read needs 32 pages. A 128 KiB write finishing in 29 µs implies the SSD acknowledges writes from a fast buffer before flash programming. Random 4 KiB writes are slower than reads and have a heavy tail (p99 181 µs), *hypothesis:* because APFS is copy-on-write, so each overwrite allocates blocks and updates metadata, while large sequential writes allocate contiguously. The sequential-write throughput (2.5 GB/s) is partly limited by fio generating 128 KiB of random data per IO outside the measured latency (IOPS x latency gives 0.56), so the throughput understates the device; the latency is unaffected.

3. Exp 2: block-size sweep

Hypothesis. Read throughput saturates at 5 to 7 GB/s; latency flat to 16 KiB then roughly linear; random approaches sequential from 128 KiB; writes saturate at 2 to 5 GB/s.

Table 2. Exp 2 results. Fixed: effective QD16 (16 psync jobs), 32 GiB range, direct I/O. Varied: block size, random vs sequential, read vs write. Mean (sd) of 3 trials. A dash means not run.

Block	MB/s: rand read	seq read	rand write	seq write	mean µs: rand read	seq read	rand write	seq write
4 KiB	778 (4)	854 (14)	192 (8)	185 (1)	84 (0)	77 (1)	340 (13)	353 (3)
16 KiB	2,753 (21)	2,866 (9)	1,560 (3)	1,506 (82)	95 (1)	91 (0)	160 (0)	166 (9)
32 KiB	4,616 (4)	4,853 (19)	3,191 (23)	3,249 (20)	113 (0)	108 (0)	150 (1)	147 (1)
64 KiB	7,369 (4)	7,840 (55)	6,638 (158)	6,796 (288)	142 (0)	134 (1)	135 (3)	133 (6)
128 KiB	10,591 (11)	10,886 (1,801)	10,736 (847)	9,330 (519)	198 (0)	196 (36)	168 (15)	198 (13)
256 KiB	12,872 (24)	11,710 (1,647)	10,382 (2,232)	9,954 (523)	326 (1)	363 (52)	371 (103)	375 (23)
512 KiB	-	10,629 (1,080)	-	10,562 (266)	-	794 (78)	-	701 (19)
1024 KiB	-	12,796 (827)	-	10,589 (1,431)	-	1,315 (88)	-	1,393 (199)

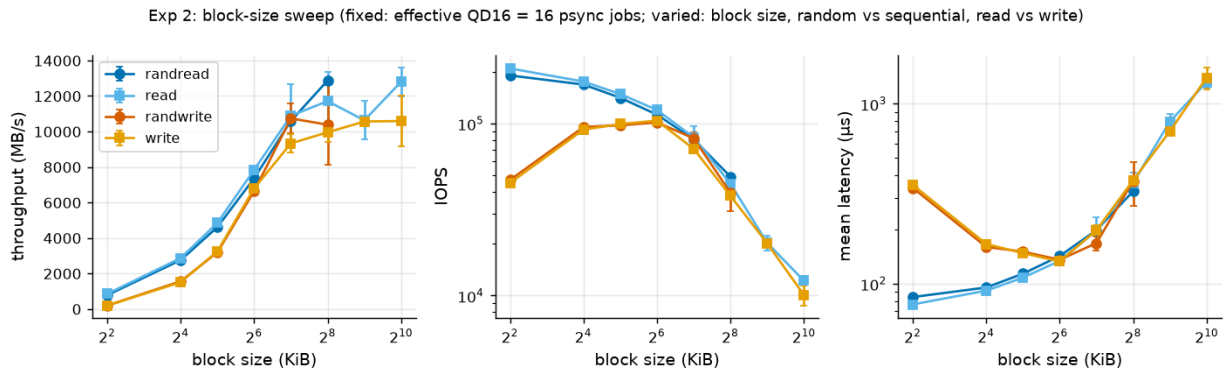


Figure 2. Exp 2: throughput (MB/s), IOPS (log) and mean latency (µs, log) against block size. Fixed: QD16. Varied: block size, pattern, direction. Error bars: sd of 3 trials.

Result. Reads saturate at 10.6 to 12.9 GB/s and writes at 9.3 to 10.7 GB/s, both far above the predicted ranges. Random and sequential differ by only 9% at 4 KiB (778 vs 854 MB/s) and are within noise from 128 KiB. Write IOPS is not monotonic: 4 KiB writes reach 45 to 47 k IOPS while 16 to 64 KiB writes reach 92 to 104 k.

Interpretation. *Inferred:* the large-block points are bandwidth-bound, and then latency follows from the queue: 16 x 256 KiB / 12,872 MB/s = 326 µs, the measured 326 µs; 16 x 1 MiB / 12.8 GB/s = 1.31 ms against 1.315 ms measured. 12.9 GB/s exceeds the PCIe 4.0 x4 payload limit (7.9 GB/s), so the link is faster than that. Random equals sequential because flash has no seek cost and direct I/O removes read-ahead. *Hypothesis:* the jump in write IOPS between 4 and 16 KiB suggests a native 16 KiB page, with 4 KiB writes being partial-page operations. Write points of 32 KiB and above show an effective depth of 13.7 to 14.6 rather than 16 (flagged, **measured**): *inferred* to be fio's own per-IO data generation outside the measured latency, so the device sees about 14 outstanding writes. Read points give 16.0.

4. Exp 3: read/write mix

Hypothesis. Mixed total IOPS below a linear blend of the pure cases; read latency 1.3 to 3 times higher at 50/50.

Table 3. Exp 3 results. Fixed: 4 KiB random, QD16, 32 GiB range. Varied: read share. Mean of 3 trials; latencies are mean / p99 in µs per operation type.

read/write %	read IOPS	write IOPS	total IOPS (sd)	total MB/s	read mean / p99 µs	write mean / p99 µs
100/0	220,362	0	220,362 (2,766)	903	72 / 108	-
70/30	78,637	33,653	112,290 (3,306)	460	134 / 1297	160 / 1341
50/50	40,454	40,511	80,964 (2,752)	332	179 / 1330	215 / 1395
0/100	0	49,417	49,417 (3,054)	202	-	323 / 1461

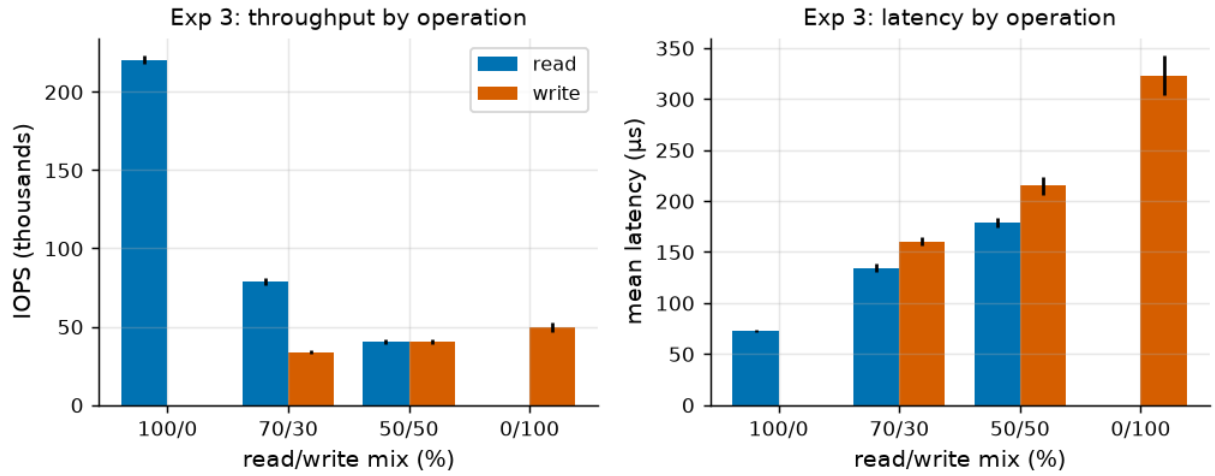


Figure 3. Exp 3: IOPS and mean latency by operation type against read/write mix. Fixed: 4 KiB random, QD16. Varied: read share.

Result. Read mean latency at 50/50 is 2.5 times the pure-read value (72 to 179 µs) and read p99 rises 12 times (108 to 1330 µs). Total IOPS is far below the arithmetic blend (135 k at 50/50) but equals the time-weighted blend: 1 / (0.5/220.4 k + 0.5/49.4 k) = 80.7 k predicted against 81.0 k measured, and 108 k against 112.3 k at 70/30.

Interpretation. *Inferred:* a read costs about 4.5 µs of device time and a write about 20 µs, and a mix averages those costs, so no extra interference penalty is needed to explain throughput. Interference shows in the tail instead: reads queue behind slow writes in the shared 16-deep queue. Write mean latency drops in a mix (215 µs at 50/50 against 323 µs pure) because fewer writers are in flight (about 9 by IOPS x latency).

5. Exp 4: queue-depth sweep and the knee

Hypothesis. IOPS rises about linearly to QD8 to 16, saturates at 180 to 250 k, knee (smallest depth reaching 90% of peak IOPS) at QD8 to 32, latency then grows as depth divided by peak IOPS.

The sweep was run twice, early (A) and again about 54 minutes later (B), because the identical workload had given two different speeds (Section 10).

Table 4. Exp 4 results, 4 KiB random read, 32 GiB range. Fixed: direct I/O, 20 s runs. Varied: effective queue depth, engine, run (A early, B later). Mean of 3 trials. The check IOPS x mean latency matched the configured depth within 1% for every psync point and within 2.1% for every posixaio point.

QD	A psync IOPS (k)	A posixaio IOPS (k)	A posixaio mean µs	B psync IOPS (k)	B posixaio IOPS (k)	B posixaio mean µs	Little's QD (A posixaio)
1	14.2	13.2	76	18.0	16.0	62	1.0
2	27.1	25.4	78	33.6	31.3	63	2.0
4	55.3	52.6	76	69.9	62.7	63	4.0
8	108.7	96.6	82	133.7	118.0	67	7.9
16	187.1	151.4	104	219.8	164.8	95	15.8
32	-	182.2	174	-	205.3	153	31.7
64	-	189.9	335	-	214.1	296	63.7
128	-	189.3	675	-	222.7	573	127.7
256	-	189.4	1,347	-	220.6	1,154	255.1

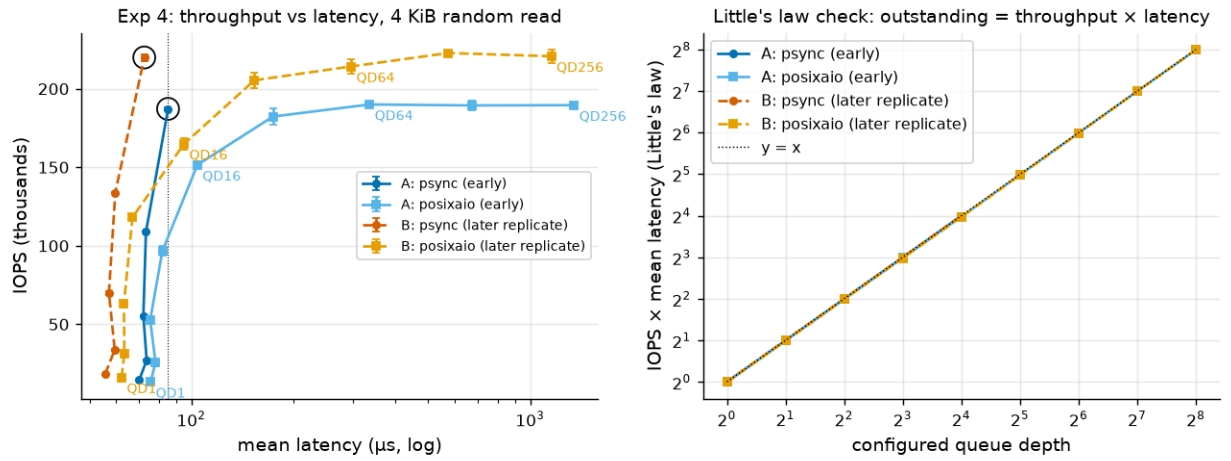


Figure 4. Exp 4: IOPS against mean latency (left, log x) and the effective-depth check (right). Fixed: 4 KiB random read. Varied: queue depth 1 to 256, engine, run A or B. Ringed points are the knee (QD16). Error bars: sd.

Result. Figure 4's left panel is the throughput-versus-latency curve (IOPS against mean latency, one point per queue depth, with curves for each engine and run) and the knee is ringed at QD16. Peak IOPS is 189.9 k (A) and 222.7 k (B). The knee by the stated rule is **QD16** in both runs: `psync` QD16 reaches 98.5% of A's peak and 98.7% of B's, while QD8 reaches only 57% and 60%. Past the knee latency is proportional to depth: 256 / 189.4 k = 1.351 ms against 1.347 ms measured (A) and 256 / 220.6 k = 1.160 ms against 1.154 ms (B).

Little's law is a check here, not an explanation. In a closed-loop fio run each of N workers issues its next IO only after the previous one completes, so IOPS x mean latency equals N nearly by construction. Its value is as a validity check that the effective queue depth is real: it confirms that `psync` with `numjobs=N` and `posixaio` with `iodepth=N` (up to 256) keep N IOs outstanding, and it shows when they do not (`psync` with `iodepth=64` gives 1.00 because a synchronous engine collapses to QD1, Section 8). It says nothing about why throughput stops growing.

The knee comes from saturation (inferred). Evidence: (1) IOPS rises almost linearly to QD8 (7.6 times the QD1 rate), bends at QD16 (13.1 times) and is flat from QD64 to QD256 (A: 189.9 k, 189.3 k, 189.4 k; B: 214.1 k, 222.7 k, 220.6 k) while latency grows with depth; (2) two different submission mechanisms end at the same ceiling, so it is not one engine's overhead (`psync` is faster below QD32, 187 k against 151 k at QD16 in A,

hypothesis: because `posixaio` hands each IO to a small kernel thread pool); (3) the limit is not interface bandwidth: the same drive moves 12.9 GB/s with 256 KiB reads at only 49 k IOPS, against 0.78 GB/s at 190 k IOPS for 4 KiB, so the 4 KiB ceiling is a per-command limit (controller command rate, flash parallelism and the host kernel path); (4) fio uses about 2.2 of the 18 cores' worth of CPU, so the host is not out of CPU. **Not established:** how the ceiling splits between controller, firmware and host path. The 17% shift between the two machine states (Section 10) shows the ceiling is not a pure device property.

6. Exp 5: tail latency

Hypothesis. At mid depth (QD4) p99 is 1.5 to 2.5 times p50 and p99.9 is 3 to 8 times p50; the spread is wider at the knee.

Table 5. Exp 5 results, 4 KiB random read, 60 s runs. Fixed: direct I/O, 32 GiB range. Varied: queue depth (4 mid, 16 knee). Mean (sd) over 3 trials; latencies in µs.

QD	IOs/run	samples above p99.9	p50 µs	p90 µs	p95 µs	p99 µs	p99.9 µs	p99.99 µs	max µs	p99.9 / p50
QD4 (mid)	3.3 M	3,347	70.1 (0.0)	79.4 (0.0)	82.4 (0.0)	100.5 (2.6)	154.5 (23.1)	844.5 (481.2)	12,979	2.20
QD16 (knee)	11.3 M	11,277	82.8 (0.6)	98.5 (1.6)	108.4 (2.1)	132.6 (4.6)	197.6 (33.8)	672.1 (432.8)	6,133	2.39

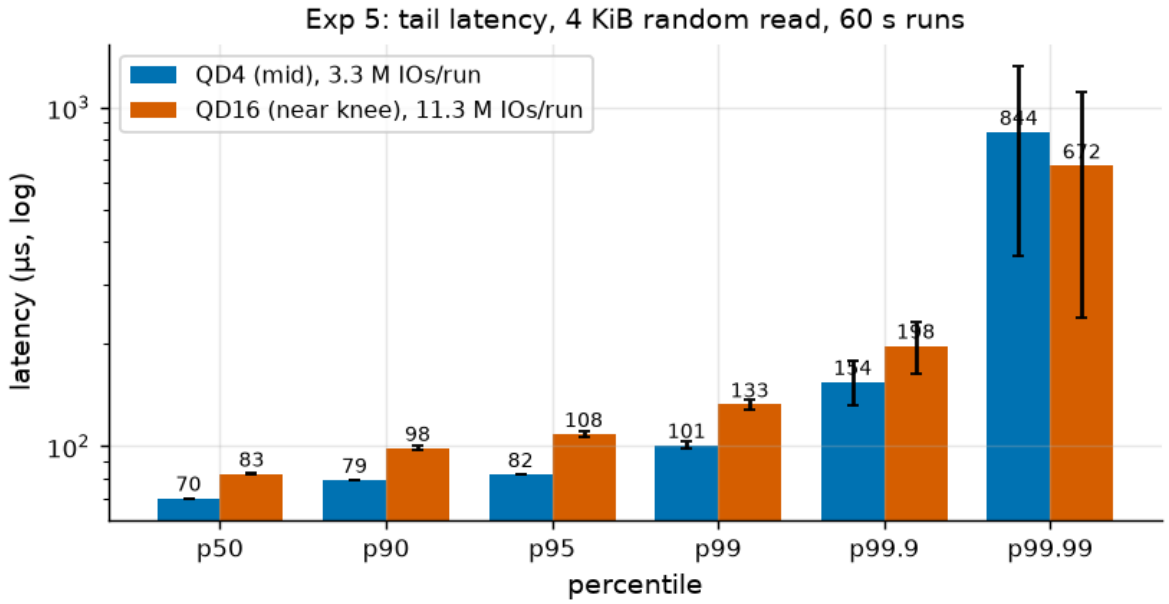


Figure 5. Exp 5: latency percentiles (µs, log scale) at QD4 and QD16, with IOs per run. Fixed: 4 KiB random read, 60 s. Varied: queue depth. Error bars: sd of 3 trials.

Result. There are 3.3 M IOs per run at QD4 (about 3,300 samples above p99.9) and 11.3 M at QD16 (about 11,300). p99/p50 is 1.43 at QD4 and p99.9/p50 is 2.2, well below the predicted 3 to 8 times: the tail is tighter than expected. The knee run's slightly larger p99.9/p50 (2.39) is within one sd of p99.9 (23 to 34 µs), so no claim is made. p99.99 rests on only 300 to 1,100 samples per run and its sd (430 to 480 µs) is more than half its mean (670 to 840 µs), so it is reported but not interpreted.

Interpretation. *Inferred:* moderate-depth reads have few interference sources (no writes, no garbage collection), so the tail is mostly queueing: from QD4 to QD16, p50 rises 18% (70 to 83 µs) and p99 32% (100 to 133 µs). The 6 to 13 ms maxima are rare outliers, *hypothesis:* OS scheduling or controller housekeeping. This experiment ran in the slower machine state. For a latency-sensitive application this means that 999 of every 1,000 4 KiB reads finished within about 150 µs at QD4 and about 200 µs at QD16 (154.5 and 197.6 µs in Table 5), 2.2 to 2.4 times the median, so a budget of roughly 150 to 200 µs per read covers all but the rarest reads; the 6 to 13 ms maxima are the exception to plan for.

7. Exp 6: working set and device state

Hypothesis. A 1 GiB range matches the 32 GiB range within 10% (reads) and 20% (writes). Sustained writes burst at 4 to 6 GB/s and may drop after cache exhaustion (about 50% chance).

6a, address range (4 KiB random, QD16, 3 trials). Reads: 189.1 k IOPS on 1 GiB against 189.6 k on 32 GiB (0.3% apart). Writes: 46.9 k against 47.8 k (1.8%). The hypothesis holds with a much smaller gap than allowed. *Inferred:* flash read and program costs do not depend on the logical range, and mapping-cache capacity is not limiting at 32 GiB.

Table 6. Exp 6b: sustained sequential write, one run. Fixed: 1 MiB blocks, 4 jobs, direct I/O, stop at 2.2 TB or 300 s. Throughput from 1 s log windows.

metric	value
fio bytes written (io_bytes)	2.199 TB
SMART host bytes written delta	2.199 TB
duration (1 s log windows)	198 s
mean throughput, first 10 s	11.69 GB/s
mean throughput, 10 to 60 s	10.99 GB/s
mean throughput, last 30 s	11.04 GB/s
slowest 1 s window	8.96 GB/s
p99.9 / max single-write latency	4,047 µs / 19.6 ms
SMART temperature start to end	34 to 65 °C (max 65)

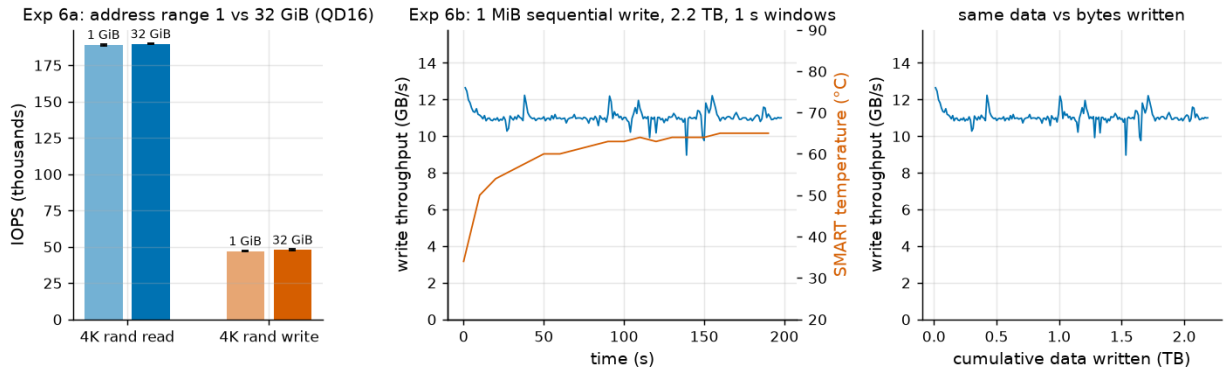


Figure 6. Exp 6: IOPS for 1 GiB and 32 GiB ranges (left, 3 trials, error bars sd); sustained write throughput in GB/s over time with SMART temperature (middle) and against bytes written (right). Fixed: 4 KiB QD16 (6a) and 1 MiB 4 jobs (6b). Varied: address range (6a), time (6b).

Result and caveat. No cliff was observed. Throughput fell from 12.6 GB/s in the first second to about 11.0 GB/s by 10 to 15 s (a 13% initial burst effect) and stayed there for the rest of the 198 s run and 2.2 TB (slowest 1 s window 8.96 GB/s; last 30 s 11.04 GB/s against 10.99 GB/s for 10 to 60 s). Also checked: p99.9 write latency 4.0 ms and maximum 19.6 ms (no stalls), temperature (up to 65 °C with no throughput change) and fio against SMART byte counts. **This is a single run, and the absence of a cliff reflects the limits of the test (the drive was never filled, and the 2.2 TB went to one 32 GiB logical range), not proof that no SLC-style cache exists.** *Hypothesis:* APFS copy-on-write sends new blocks to the SSD while old ones are trimmed, so the drive always had free blocks; exposing a cliff would need the drive to be filled, which the 20% free-space rule forbids. The byte cap replaced a 300 s-only plan that would have written about 3.3 TB at the measured rate.

8. Exp 7: misleading setup (page cache and wrong queue depth)

Hypothesis. Buffered reads show 1 to 5 µs and 50 to 300 times the IOPS of direct reads; `psync` with `iodepth=64` behaves like QD1.

Table 7. Exp 7 results, 4 KiB random read on a 1 GiB range. Fixed: bs 4 KiB, 20 s + 5 s ramp. Varied: setup A to D. Mean (sd) over 3 trials.

setup	IOPS (sd)	mean µs (sd)	p99 µs (sd)	configured QD	effective QD (Little)
A: direct=1, QD1 (correct)	14,540 (230)	68.63 (1.06)	81.41 (2.05)	1	1.00
B: direct=0 buffered, QD1 (flawed: page cache)	1,316,919 (79,092)	0.69 (0.05)	1.00 (0.00)	1	0.90
C: direct=1, QD16 (correct)	191,798 (504)	83.19 (0.21)	125.10 (0.59)	16	15.96
D: psync with iodepth=64 (flawed: depth ignored)	14,395 (42)	69.35 (0.20)	81.75 (1.56)	64	1.00

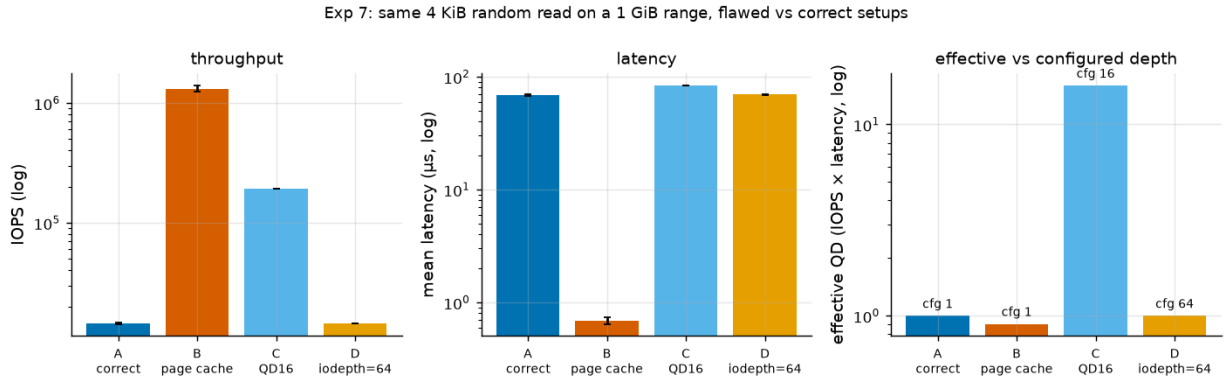


Figure 7. Exp 7: IOPS (log), mean latency (log) and effective depth (log) for setups A (direct, QD1, correct), B (buffered, QD1, page cache), C (direct, QD16, correct) and D (psync with iodepth=64). Varied: setup. Error bars: sd.

Result and mechanism. Buffered reads (B) give 1.32 M IOPS at 0.69 µs, **90.6 times** the IOPS and 99 times lower latency than the correct direct run (A: 14.5 k IOPS, 68.6 µs). That is 5.4 GB/s of "reads" from a drive whose real QD1 random-read rate is 60 MB/s, and it is 6 times the best real 4 KiB rate (223 k IOPS, C). *Measured:* with the same file and job and only `direct` changed, the data come from RAM. *Inferred:* buffered `pread` hits the unified buffer cache and never reaches the NVMe queue. The latency is below the predicted 1 to 5 µs because fio recorded a minimum of 0 ns, so many hits complete faster than its timer resolves

(*inferred*). Setup D is identical to A (14,395 against 14,540 IOPS) and its effective depth is 1.00 against 64 configured: the number in the job file is not the depth the device saw.

9. Hypothesis scorecard

The predictions (stated as the hypothesis of each experiment in Sections 2 to 8) were written and committed to the repository's git history before any experiment data were collected: commit 4f2fee8, 2026-10-16 11:32:40 -0400. The only earlier measurements were 5-second engine checks that test whether queue depth is real; the predictions disclose them. Two dated amendments were committed together in commit 69b59d1, 2026-10-22 19:17:35 -0400, before the first experiment run (19:25 the same day). **Amendment A** records that smoke tests changed the design: fio cannot run two 16-job processes at once, the knee rule is applied across all depth points that pass the effective-depth check, and the sustained write is capped at 2.2 TB or 300 s (the drive writes at about 12 GB/s) with Exp 2 write runs shortened to 4 s. **Amendment B** records that the macOS upgrade from 26.5.2 to 27.0.1 raised the asynchronous-I/O limits, so the prediction that `posixaio` stays near depth 16 was withdrawn, Exp 4 was redesigned (`psync` depths 1 to 16 plus `posixaio` depths 1 to 256) and the Exp 7 wrong-depth example became `psync` with `iodepth=64`. The predictions were not edited after the amendments.

Table 8. Predictions scored against results.

Exp	Prediction	Outcome
1	4K random read 60 to 90 µs	Missed low in Exp 1 (54.6 µs); in range in the other machine state (68.6 to 70.1 µs)
1	4K random write faster than read	Wrong (80.9 vs 54.6 µs)
1	128K sequential read 70 to 150 µs	Hit (135 µs)
1	128K sequential write 80 to 200 µs	Missed (29 µs)
2	Read plateau 5 to 7 GB/s	Missed (10.6 to 12.9 GB/s)
2	Write plateau 2 to 5 GB/s	Missed (9.3 to 10.7 GB/s)
2	Random approaches sequential from 128K	Hit (and within 9% at 4K)
3	Mix total below linear blend	Hit, but explained by time-weighting, not interference
3	Read latency 1.3 to 3 times at 50/50	Hit (2.5 times mean, 12 times at p99)
4	Saturation 180 to 250 k IOPS, knee QD8 to 32	Hit (189 k and 223 k, knee QD16)
4	Effective depth matches configured within 5% (psync)	Hit (within 1%)
4	posixaio stays near depth 16	Withdrawn in Amendment B before data collection (true only on macOS 26.5.2)
5	p99.9 is 3 to 8 times p50	Missed (2.2 to 2.4 times)
6a	Range within 10% (read), 20% (write)	Hit (0.3%, 1.8%)
6b	Burst 4 to 6 GB/s, possible cliff	Burst far higher (11.7 GB/s over 10 s); no cliff
7	Buffered 50 to 300 times the IOPS	Hit (90 times)
7	`iodepth` ignored by `psync`	Hit (effective depth 1.00)

10. Anomalies

1. Two performance levels for an identical workload. 4 KiB random read at 16 `psync` jobs gave 187 to 192 k IOPS in Exp 2, 4, 5, 6a and 7, and 215 to 224 k IOPS in Exp 3, a drift check, a 30-run time series and the Exp 4 replicate (Figure 8). The difference is visible at QD1 too (55.6 µs against 70.1 µs mean), so it is not only thread placement at saturation. In the high level fio's kernel CPU use is about 262% of a core against about 209% (**measured**; a consequence, not a cause). **Analysis of existing data only, no new runs:** the 57 runs of this workload (15 low, 42 high) form four consecutive episodes (low 19:32, high 22:32, low 22:43 to 23:13, high 23:21 to 23:38), so each factor is effectively tested on four episodes.

Table 9. Factors tested against the two IOPS levels (57 runs, existing metadata). r is the point-biserial correlation with the level.

Candidate factor	Low level (~189 k)	High level (~222 k)	Point-biserial r with level
Run order (index 0 to 56)	mean 9.4	mean 34.6	+0.68
Minutes since last write run of at least 10 GB	mean 23.8 (min 0.3)	mean 6.2 (min 0.3)	-0.68
GB written in the 30 min before the run	mean 307.9	mean 2136.3	+0.92
Power source at run start (pmset log)	AC in 15 of 15 runs	AC in 42 of 42 runs	not computable (no variation) (battery charge held at [77, 80] %)
Display on at run start (user activity proxy)	on in 12 of 15	on in 42 of 42	+0.39
Sleep events during the run	0	0	none
SSD temperature (SMART, stage start and end, C)	[28, 40, 41, 43, 46, 53, 58]	[37, 41, 42, 43, 46, 50, 58]	overlap, no per-run reading
fio system CPU (% of one core, summed)	mean 209.4	mean 262.2	+0.99 (consequence)

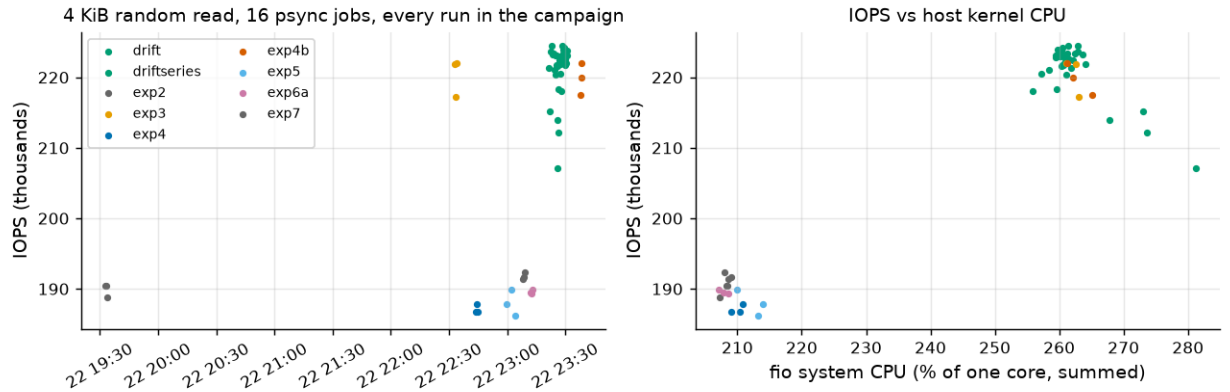


Figure 8. Every run of the identical workload (4 KiB random read, 16 psync jobs, 32 GiB range) against time, coloured by experiment (left), and IOPS against fio system CPU (right). Fixed: workload. Varied: time of run.

Power source: AC for all 57 runs, so there is no variation to explain anything. Sleep: none during any run. Thermal: SMART temperature exists only at stage boundaries, the ranges overlap between levels, and no per-run reading exists, so there is no evidence either way. Display on: 12 of 15 low runs against 42 of 42 high runs, a weak proxy for user activity. Run order correlates (r = +0.68) only through the episode layout. The strongest association is the volume written in the 30 minutes before the run (r = +0.92: mean 2,136 GB before high runs, 308 GB before low runs), but it is confounded with run order and one episode contradicts it: Exp 4's QD16 runs at 22:43 were low although 1.5 TB had been written about 12 minutes earlier. **Conclusion: no single factor accounts for the split; recent write volume is the only candidate that lines up,**

and it is an untested hypothesis. Other **untested hypotheses:** a storage or SoC power or clock state; macOS placement of fio threads on Super or Performance cores; background system activity. **Consequence:** absolute numbers from different experiments carry about 17% uncertainty; comparisons within one experiment hold.

2. Invalid Exp 2 write runs: the Mac slept during the first pass (Section 1); quarantined and repeated. **3. OS upgrade mid-project:** macOS 26.5.2 to 27.0.1 raised `kern.aiomax` from 90 to 1024 and `kern.aioprocmax` from 16 to 256, so the first conclusion that `posixaio` is capped at 16 became wrong; the engine check was repeated and the predictions were amended (Amendment B, Section 9) before any experiment data. **4. Writes show an effective depth of 13.7 to 14.6** at 16 jobs: fio's data generation sits outside the measured latency. **5. 4 KiB writes have half the IOPS of 16 KiB writes:** *hypothesis:* partial-page writes. **6. Exp 3 and 4 SMART read delta is 1.55 times fio's count** against about 1.15 expected from ramp time: unexplained.

11. Limitations

- **One machine, one file, one filesystem.** Controller, FileVault encryption, APFS copy-on-write and OS cache layers cannot be disabled, so "SSD performance" means file-on-APFS performance.
- **F_NOCACHE direct I/O is not a durability test (no F_FULLFSYNC):** write latencies are acknowledgements from the device buffer and say nothing about power-loss behavior.
- **macOS has no blktrace or Linux-style `iostat`.** Observable: fio latency histograms, SMART counters, `pmset` logs, per-process CPU. Not observable: controller-internal queue depth, per-run SSD temperature, PCIe link state, garbage-collection or SLC state, and F_NOCACHE at syscall level.
- **fio runs at most about 16 jobs here**, so `psync` depths stop at 16; deeper depths use `posixaio`, whose latency includes thread hand-off overhead.
- **The sustained write is one run and never filled the drive.** Exp 2 write runs are 4 s short-window means; Exp 6b is the steady-state check and agrees with their plateau.
- **The two machine states (Section 10)** limit absolute cross-experiment comparison to about 17%.

12. Interface and device ceilings against measured

Table 10. Roofline-style summary. Measured values are the trial means reported in this document; PCIe payload ceilings are public specification values (link generation assumed, not read).

Quantity	Measured	Reference
4 KiB random read IOPS, QD16 to QD256	189 k (A) to 223 k (B)	0.8 to 0.9 GB/s, 6 to 7% of best read bandwidth
Best read bandwidth (256 KiB random)	12.9 GB/s	PCIe 4.0 x4 payload 7.9 GB/s; 5.0 x4 15.8 GB/s (link not readable on macOS)
Best write bandwidth (1 MiB sequential)	10.6 GB/s short runs; 11.0 GB/s sustained over 2.2 TB	about 82 to 86% of the read ceiling
QD1 4 KiB random read latency	54.6 µs (Exp 1) to 70.1 µs (Exp 4)	flash page read is tens of µs; the rest is controller, encryption, protocol and file system

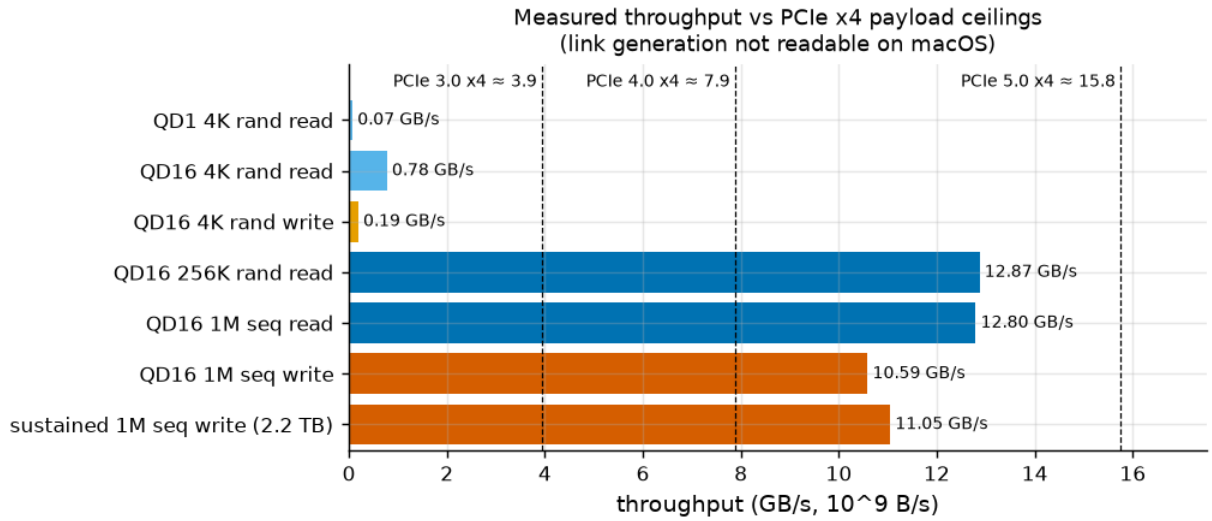


Figure 9. Measured throughput (GB/s) against PCIe x4 payload ceilings. Fixed: direct I/O, 32 GiB file. Varied: workload.

Inferred: 12.9 GB/s exceeds PCIe 4.0 x4 (7.9 GB/s), so the drive uses a Gen5-class or Apple-specific link, and reads reach 82% of a Gen5 x4 payload ceiling, so a link-bound read ceiling is plausible. Small reads are IOPS-bound, not bandwidth-bound: 4 KiB at 190 to 223 k IOPS uses about 6 to 7% of the read bandwidth.

13. Reproducibility

Everything is in the repository folder `Project-3`: `fio/` (job files), `scripts/` (drivers and analysis), `data/raw/` (untouched fio JSON, health logs and the quarantined runs), `data/processed/` (CSVs and generated tables) and `plots/` (figures). One command reproduces the full study: `brew install fio smartmontools`, then `bash scripts/run_all.sh`. It waits for AC power, runs the engine check, preconditions the 32 GiB file, runs the direct I/O check (stopping if `direct=1` looks cache-like), then Exp 1 to 7, a drift series and the Exp 4 replicate, and rebuilds tables and figures. Conditions: charger plugged in, lid open, machine idle. A clean run takes about 3 hours and writes about 4 TB; this project's SMART counter recorded about 6.4 TB in total, including invalidated runs and smoke tests. `bash scripts/cleanup.sh` deletes only the test file and marker files.

Appendix A. AI-use note

- **Authorship:** Three students, each responsible for a group of experiments, the matching report sections and the checks behind them. Tiffany Cheng: safety guards, preconditioning, Exp 1 (QD1 baselines) and Exp 2 (block size). Marie Morales: Exp 3 (read/write mix), Exp 4 (queue-depth sweep) and the knee analysis. Alex Redd: Exp 5 (tail latency), Exp 6 (device state, working set), Exp 7 (misleading setup) and the anomalies. The team supplied the experiment list, the safety rules and the approvals: installing fio and smartmontools, a direct against buffered check before any full run, a 5-minute cap on the sustained write, 3 trials for the tail experiment, the corrected write-volume estimate, and the stage order. The students do not claim to have written the code.
- **Tool:** Claude Code generated the fio job files, run and parse scripts, plotting code, written predictions and amendments, documentation and the first drafts of this report, and it ran the benchmarks through the driver scripts.

- **Problems found during development, each of which changed the method:**
    - The first write-volume estimate of 150 to 300 GiB was far too low. The project wrote about 6.4 TB because the drive writes at 10 to 12 GB/s. The team corrected it.
    - Claude Code found a stale engine conclusion from smoke data: posixaio was believed capped at 16, but after the macOS upgrade the limit was 256. The check was repeated and the change is recorded in Amendment B (Section 9) before any experiment data.
    - Claude Code also found invalid Exp 2 write runs, because the driver checked AC power only at stage start and did not prevent sleep. Per-run power checks, caffeinate and a wall-time validity check were added, and the runs were repeated.
    - Shell bugs made scripts exit silently ($rw__bs parsed as one variable; two set -e with pipefail cases).
- **Where expectations were wrong**: Writes slower than reads at QD1, a tighter tail than predicted with p99.9 at 2.2 times p50 against 3 to 8 predicted, and read and write bandwidth about 2 times higher than predicted), the outcomes are scored as such in Section 9 and the predictions were not adjusted.
- **Verification:** The parser was checked by hand against the raw fio JSON, and every table is generated from the CSVs. An automated script in the repository traces every number in the report to measured data, raw logs or an explicit formula. Independent checks recorded in the repository: direct against buffered latency (87 times, with a stop rule), SMART read and write counters against fio byte counts, rerun of the sleep-affected Exp 2 runs, and the psync iodepth=64 check (effective depth 1.00). Not verified: the explanations labeled hypothesis and the cause of the two performance levels.